



Chapter S4D: Web App Flask

CONTENT

No.	Topic	Syllabus Guide
1	Web and Native Applications	4.2.1
2	Usability Principles in Web Applications	4.2.2
3	Dynamic Web Pages	4.2.3 – 4.2.4
4	The Flask Framework	
5	HTTP Requests and Routing in Flask	
6	HTTP Responses and Status Codes	
7	Templates and Rendering	
8	Static Files	
9	Processing Form Data	
10	Handling File Uploads	
11	Summary of Request Data in Flask	
12	Programming Exercises	



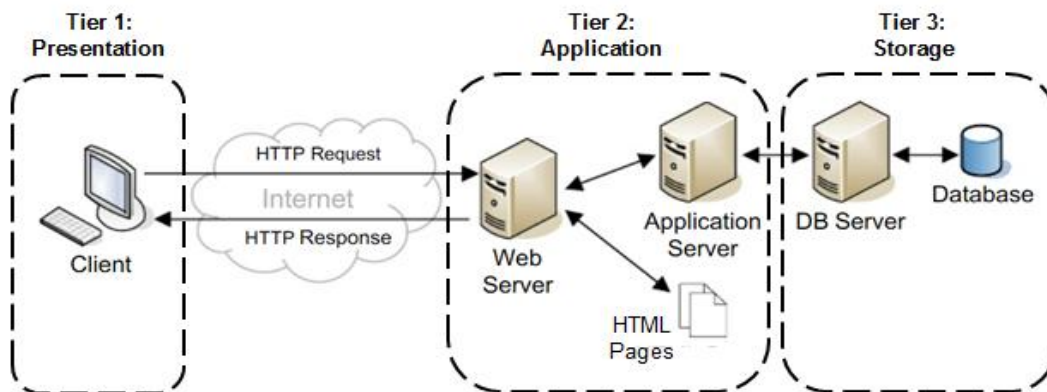
1. Web and Native Applications

A **native application** is a software application installed on a device and designed for a specific operating system. Examples include WhatsApp (mobile app), Shopee (mobile app).

A **web application** is a client-server application that runs in a web browser. Examples include web-based email, online office applications, browser-based e-commerce platforms, and browser-based social media.

Applications are usually structured into tiers, each with a specific role. Web applications commonly use a three-tier architecture:

- **Presentation tier:** Handles user interface (web browser)
- **Application tier:** Processes requests and contains business logic
- **Storage tier:** Stores and retrieves data (database)



The web browser sends HTTP requests to the application tier, which processes them, interacts with the database on the server through queries, and returns responses. This **separation of concerns** improves scalability, maintainability, and security. Developers must design how these tiers communicate effectively.

Comparison of Native and Web Apps:

Feature	Native Apps	Web Apps
Platform	Specific operating system	Platform-independent (browser-based)
Installation	Installed on device	No installation required
Performance	Faster (local execution)	Slower (network latency)
Connectivity	Can work offline	Usually requires internet
Updates	Manual updates required	Updated centrally on server

Quick Check:

1. Define a web application.
2. State the three tiers in a web application architecture.
3. Explain how a user request is processed in a web application.
4. Give two differences between web and native applications.
5. State one advantage of web applications over native applications.



2. Usability Principles in Web Applications

Web applications are designed to provide information and services to users from different backgrounds. **Usability** ensures the system is easy, efficient, error-free, memorable, and satisfying. It can be defined by five quality components:

- **Learnability:** How easily a new user can perform basic tasks.
Example: Intuitive menus on a shopping site.
- **Efficiency:** How quickly users can complete tasks once they know the system.
Example: Keyboard shortcuts in an online office app.
- **Memorability:** How easily users can remember the system after a period of not using it. *Example:* Navigation layout remains familiar after a month.
- **Errors:** How the system prevents mistakes and helps users recover from them.
Example: Clear error messages when login fails.
- **Satisfaction:** How pleasant and satisfying the system is to use.
Example: Attractive and responsive interface for social media apps.

(Reference: <https://www.nngroup.com/articles/usability-101-introduction-to-usability/>)

3. Dynamic Web Pages

Static web pages always serve the same content (HTML/CSS) for every user request. **Dynamic web pages** customise the content for each request, e.g., showing random messages, user-specific data, or different styles.

Without a framework, programmers must manually handle HTTP requests and responses, construct HTML/CSS strings in code, and ensure the HTTP standard is followed. This is **tedious** and **error-prone**, especially for complex web applications. **Using Flask** (Python web framework) automates request handling, routing, and response generation. Flask simplifies creating dynamic web pages by handling HTTP details for the programmer, so that they can focus on business logic and templates.

Example: Display a random message with a random text colour each time the page loads:

```
from flask import Flask, render_template_string
import random

app = Flask(__name__)

MESSAGES = ["Hello, World!", "Computing is Fun", "Alamak!"]
COLOURS = ["red", "blue", "green", "orange", "purple"]

@app.route("/")
def home():
    msg = random.choice(MESSAGES)
    colour = random.choice(COLOURS)
    return render_template_string(f"<p style='color:{colour}'>{msg}</p>")

if __name__ == '__main__':
    app.run()
```



4. The Flask Framework

Python frameworks like **Flask** and Django make creating dynamic web applications easier. Flask provides a basic web server that handles HTTP automatically. Here is a minimal setup:

```
from flask import Flask

app = Flask(__name__)

if __name__ == '__main__':
    app.run() # Default port 5000
```

Visiting `http://127.0.0.1:5000/` shows **404 Not Found** until routes are defined. You can change the port: `app.run(port=12345)`. Flask handles HTTP details for you, so you can focus on routes, templates, and logic.

5. HTTP Requests and Routing in Flask

Flask uses routes to map HTTP request to Python functions. Each route corresponds to a **path** (URL) and optionally an **HTTP method** (GET, POST). If a request matches a route, Flask calls the associated function to generate a response. If no route matches, a **404 Not Found** is returned. **Routes in Flask** are declared using **decorators**: `@app.route('/path')`.

5.1 Fixed Routes

A fixed route maps a single URL path to a function.

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'Welcome'

@app.route('/report')
def generate_report():
    return 'Everything is awesome'

@app.route('/readme.txt')
def readme():
    return 'READ ME'

if __name__ == '__main__':
    app.run() # Default port 5000
```

- Visiting `http://127.0.0.1:5000/` → "Welcome"
- Visiting `http://127.0.0.1:5000/report` → "Everything is awesome"
- Visiting `http://127.0.0.1:5000/readme.txt` → "READ ME"
- Any other path → 404 Not Found
- Routes are **case-sensitive**.



5.2 Trailing Slashes

Routes and request paths must match exactly, including slashes. Flask automatically redirects if a trailing slash is missing for a route ending with /.

```
@app.route('/optional_slash/')
def optional_slash():
    return 'Optional Slash'
```

- `http://127.0.0.1:5000/optional_slash` → Flask redirects to `/optional_slash/`
- `http://127.0.0.1:5000/optional_slash/` → "Optional Slash"
- `http://127.0.0.1:5000/optional_slash/extra` → 404 Not Found

5.3 Multiple Decorators

One function can handle multiple routes using the multiple `@app.route()` decorators.

```
@app.route('/one/')
@app.route('/one/two/')
@app.route('/three/two/one')
def multiple():
    return 'Handled by multiple()'
```

- All three URLs call the same function.
- `http://127.0.0.1:5000/one/` → "Handled by multiple()"
- `http://127.0.0.1:5000/one/two/` → "Handled by multiple()"
- `http://127.0.0.1:5000/three/two/one` → "Handled by multiple()"

5.4 Variable Routes

Routes can include **variable parts** using `<variable_name>`. Default type is `str`.

```
@app.route('/string/<s>/')
def string_variable(s):
    return f'string = {s}'
```

- `http://127.0.0.1:5000/string/hello/` → "string = hello"
- `http://127.0.0.1:5000/string/123` → "string = 123"
- `http://127.0.0.1:5000/string/` → 404 Not Found
- `http://127.0.0.1:5000/string/hi/there` → 404 Not Found

5.5 Variable Routes with Converters

Flask supports different type converters to enforce types or capture complex paths:

Converter	Syntax	Description	Example URL
int	<code><int:number></code>	Only integers allowed	<code>/integer/42/</code>
float	<code><float:number></code>	Only floating-point numbers	<code>/temperature/36.7/</code>
path	<code><path:name></code>	Matches slashes / as well as other characters	<code>/files/docs/report.txt</code>



```
@app.route('/integer/<int:i>/')
def integer_variable(i):
    return f'integer = {i}'

@app.route('/temperature/<float:t>/')
def show_temp(t):
    return f'Temperature = {t}°C'

@app.route('/files/<path:file_path>/')
def show_file(file_path):
    return f'File path = {file_path}'
```

Examples for int:

- `http://127.0.0.1:5000/integer/123/` → "integer = 123"
- `http://127.0.0.1:5000/integer/0` → "integer = 0"
- `http://127.0.0.1:5000/integer/-123` → 404 Not Found
- `http://127.0.0.1:5000/integer/one` → 404 Not Found

Examples for float:

- `http://127.0.0.1:5000/temperature/36.7` → "Temperature = 36.7°C"
- `http://127.0.0.1:5000/temperature/42` → 404 Not Found
- `http://127.0.0.1:5000/temperature/abc` → 404 Not Found

Examples for path:

- `http://127.0.0.1:5000/files/docs/report.txt` → "File path = docs/report.txt"
- `http://127.0.0.1:5000/files/images/icons/logo.png` → "File path = images/icons/logo.png"
- `http://127.0.0.1:5000/files/` → 404 Not Found

5.6 Routing by HTTP Methods

Flask routes can be restricted to specific HTTP methods, such as **GET** or **POST**.

- **GET**: Retrieve data without modifying the server.
- **POST**: Submit data or make changes on the server (e.g., add, update, delete).

This ensures that operations that modify data cannot be triggered by simply visiting a URL in a browser.

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/submit_data', methods=['POST'])
def submit_data():
    return 'Data submitted successfully!'

if __name__ == '__main__':
    app.run()
```

- `http://127.0.0.1:5000/submit_data` via a browser (GET) → **405 Method Not Allowed**
- Submitting a **POST request** via an HTML form → success



Example HTML form to trigger **POST**:

```
<form action="/submit_data" method="post">
  <input type="text" name="info">
  <input type="submit" value="Send">
</form>
```

- Use `methods=[...]` in the route decorator to restrict HTTP methods.
- GET should **never** change server data.
- POST is used for adding/updating/deleting data.
- Accessing a POST-only route via GET produces a **405 error**.

Exercise 1

Complete the following program so it greets the user when the site is visited with the user's name in the path (as long as the name does not include a slash):

```
import flask
app = flask.Flask(__name__)

@app.route(      )
def home(      ):
    return 'Hello, {}'.format(      )

if __name__ == '__main__':
    app.run()
```

5.7 Generating Paths from Function Names

Flask allows you to **generate URLs programmatically** from Python function names using `url_for()`. This is useful when you want to avoid hardcoding paths in templates or redirects, or your routes have **variable parts**.

Using `url_for()`:

- The function name is passed as a string to `url_for()`.
- If the route has variables (e.g., `/string/<s>`), provide them as **keyword arguments**.
- `url_for()` returns the full path that Flask has mapped to the function.

Example: The following example specifies some routes and prints three generated paths in the shell or command prompt window when the "root" site `http://127.0.0.1:5000/` is visited.

```
from flask import Flask, url_for

app = Flask(__name__)

@app.route('/')
def home():
    url1 = url_for('fixed_route')
    url2 = url_for('string_variable', s='example')
```



```
url3 = url_for('integer_variable', i=2468)

print(url1) # Output: /fixed/
print(url2) # Output: /string/example
print(url3) # Output: /integer/2468

return 'Check your shell or command prompt window'

@app.route('/fixed/')
def fixed_route():
    return 'Routed to fixed_route()'

@app.route('/string/<s>')
def string_variable(s):
    return f'Routed to string_variable(), s = {s}'

@app.route('/integer/<int:i>')
def integer_variable(i):
    return f'Routed to integer_variable(), i = {i}'

if __name__ == '__main__':
    app.run()
```

Expected output in Shell/Command Prompt:

```
/fixed/
/string/example
/integer/2468
```

These correspond to calling `fixed_route()`, `string_variable(s='example')` and `integer_variable(i=2468)` respectively.

Exercise 2

The following program runs a web server at `http://127.0.0.1:5000/`.

```
import flask

app = flask.Flask(__name__)

NAMES = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
        'September', 'October', 'November', 'December']

@app.route('/')
def home():
    return 'Home'

@app.route('/<int:month>/')
def name_month(month):
    if month in range(1, 13):
        return 'Month {}: {}'.format(month, NAMES[month - 1])
    return 'Invalid month'
```




RAFFLES INSTITUTION

H2 Computing (9569)

2026 Year 6

```
@app.route('/compare/<float:temp>/')
def compare_temp(temp):
    if temp > 35.5:
        return 'It\'s hot!'
    if temp < 25.5:
        return 'It\'s cold!'
    return 'It\'s normal!'

@app.route('/greet/')
def greet():
    return 'Hello!'

@app.route('/greet/<name>/')
def greet_name(name):
    return 'Hello, {}!'.format(name)

@app.route('/data/', methods=['POST'])
def post_data():
    return 'You are using POST'

@app.route('/data/', methods=['GET'])
def get_data():
    return 'You are using GET'

if __name__ == '__main__':
    app.run()
```

With this program running in the background, predict the output when each of the URLs below is visited using a browser. If you predict that a HTTP error would occur instead, write "ERROR" as the output. After completing the table, visit the URLs to check your answers.

No.	URL	Output
1	http://127.0.0.1:5000/	
2	http://127.0.0.1:5000/10	
3	http://127.0.0.1:5000/10/20/	
4	http://127.0.0.1:5000/20/	
5	http://127.0.0.1:5000/compare/35.4	
6	http://127.0.0.1:5000/compare/35.6/	
7	http://127.0.0.1:5000/compare/	
8	http://127.0.0.1:5000/greet/world/	
9	http://127.0.0.1:5000/greet/worLD/	
10	http://127.0.0.1:5000/Greet/world/	
11	http://127.0.0.1:5000/greet/Mei Yi/	
12	http://127.0.0.1:5000/greet/	
13	http://127.0.0.1:5000/data/	



6. HTTP Responses and Status Codes

When a Flask route returns a string, Flask automatically creates a **complete HTTP response**.

An HTTP response consists of:

- **Status code** (e.g., 200 OK, 404 Not Found, 405 Method Not Allowed)
- **Headers** (e.g., Content-Type)
- **Body** (the content returned, usually HTML)

Example:

```
import flask

app = flask.Flask(__name__)

@app.route('/')
def index():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

The browser actually receives a response like:

```
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 13
Server: Werkzeug/0.14.1 Python/3.6.4
Date: Thu, 07 Feb 2019 03:18:45 GMT

Hello, World!
```

By default, Flask sets the Content-Type to text/html, so the return value is treated as HTML. This allows HTML tags to be used:

```
import flask

app = flask.Flask(__name__)

@app.route('/')
def index():
    return '<h1>Welcome!</h1> <b>Hello,</b> <i>World!</i>'

if __name__ == '__main__':
    app.run()
```

Although this works, it is technically non-standard. It is good practice to return full HTML documents.



6.1 Changing the Status Code

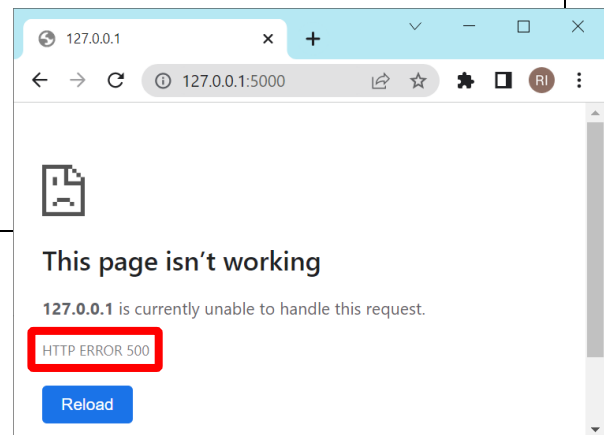
Flask returns a default HTTP status code of **200 (OK)**. This can be changed by returning a tuple, where the second value is the status code.

```
import flask

app = flask.Flask(__name__)

@app.route('/')
def index():
    return ('', 500)

if __name__ == '__main__':
    app.run()
```



6.2 Changing the Response Headers

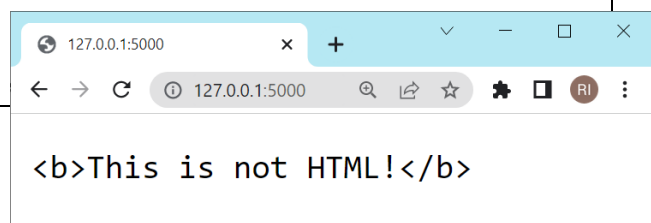
You can add response headers by using a dictionary and returning this dictionary as the **third item** of the return tuple. For instance, if we want the web browser to treat our response as plain text instead of HTML, we can replace the Content-Type header value with "text/plain" instead:

```
import flask

app = flask.Flask(__name__)

@app.route('/')
def index():
    headers = {'Content-Type': 'text/plain'}
    return ('<b>This is not HTML!</b>', 200, headers)

if __name__ == '__main__':
    app.run()
```





6.3 Redirecting to Another URL

Flask can redirect users to another URL using the `redirect()` function. This is useful when we want the browser to load a different page.

```
import flask
from flask import redirect

app = flask.Flask(__name__)

@app.route('/')
def index():
    return redirect('http://example.com')    #redirecting to example.com

if __name__ == '__main__':
    app.run()
```

To redirect to another route in the same application, use `url_for()` instead of hardcoding the URL:

```
import flask
from flask import redirect, url_for

app = flask.Flask(__name__)

@app.route('/new_url/')
def moved_index():
    return 'You have reached the new URL!'

@app.route('/')
def index():
    return redirect(url_for('moved_index'))

if __name__ == '__main__':
    app.run()
```

`url_for()` ensures the correct path is used and avoids hardcoding URLs.



7. Templates and Rendering

HTML can be generated directly in Flask using strings, but this is messy and hard to maintain.

Example of a HTML document with headings and hyperlinks returned in Flask application:

```
import flask

app = flask.Flask(__name__)

@app.route('/')
def home():
    html = '<!DOCTYPE html>\n<html>'
    html += '<head><title>Home Page</title></head>'
    html += '<body><h1>Welcome to my home page!</h1>'
    html += '<p>My favourite web site is '
    html += '<a href="http://example.com/">'
    html += 'example.com</a>.</p>'
    html += '<p>You can greet Alex '
    html += '<a href="/greet/Alex/">here</a>.</p>'
    html += '</body></html>'
    return html

@app.route('/greet/<name>/')
def greet(name):
    html = '<!DOCTYPE html>\n<html>'
    html += '<head><title>Greetings!</title></head>'
    html += '<body><h1>Hello, {}!</h1>'.format(name)
    html += '</body></html>'
    return html

if __name__ == '__main__':
    app.run()
```

Run the above program and visit <http://127.0.0.1:5000/>. Notice that a full HTML page appears, complete with a page title, headings and hyperlinks. Press Ctrl-U to verify that the HTML for the page comes from the `home()` function of the above Python program.

Now visit <http://127.0.0.1:5000/greet/Ben/>. Notice that the result has a page title and the greeting for Ben is formatted to look like a heading using the `<h1>` tag.



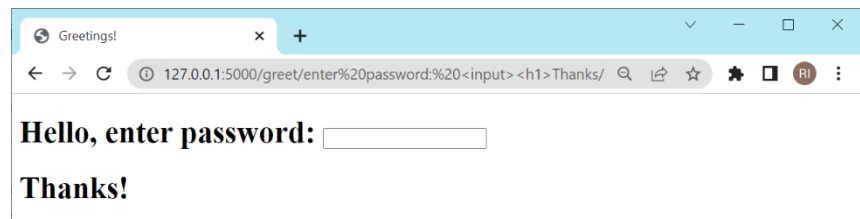
7.1 Why Use Templates?

Writing HTML directly in Python is messy and hard to maintain. It can be unsafe as a malicious user can inject HTML via the URL and potentially cause all kinds of mischief.

Example, visit the following:

`http://127.0.0.1:5000/greet/enter%20password:%20<input><h1>Thanks/`

This injects HTML that could appear as a fake form.



Flask uses **templates** to separate HTML from Python code. This makes code cleaner and easier to manage, and prevents HTML injection (unsafe user input).

7.2 The Jinja2 Template Engine

Flask uses **Jinja2** as its template engine. Templates are stored in a **templates/** folder.

Template 1: templates/home.html

```
<!DOCTYPE html>
<html>
<head><title>Home Page</title></head>
<body>
<h1>Welcome to my home page!</h1>
<p>My favourite web site is
  <a href="http://example.com/">example.com</a>.</p>
<p>You can greet Alex
  <a href="{{ url_for('greet', name='Alex') }}">here</a>.
</p>
</body>
</html>
```

Template 2: templates/greet.html

```
<!DOCTYPE html>
<html>
<head><title>Greetings!</title></head>
<body>
<h1>Hello, {{ visitor }}!</h1>
</body>
</html>
```



RAFFLES INSTITUTION H2 Computing (9569)

2026 Year 6

- Placeholders in Jinja2 use `{{ }}` to insert dynamic values.
- Jinja2 expressions are similar to Python but run in the template environment (not all Python functions are available, e.g. `len()`).
- Filling in placeholders to produce the final HTML is called **rendering**.

Rendering templates in Flask:

```
from flask import Flask, render_template

app = Flask(__name__)

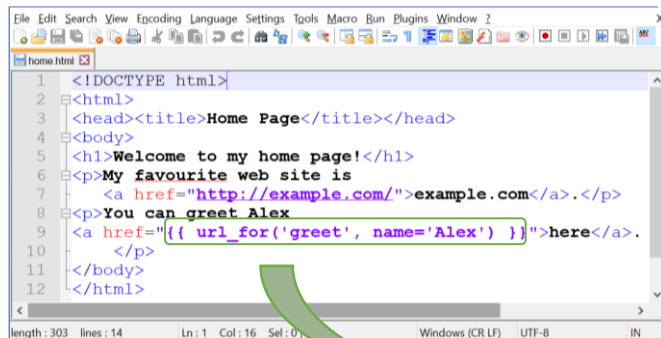
@app.route('/')
def home():
    return render_template('home.html')

@app.route('/greet/<name>/')
def greet(name):
    return render_template('greet.html', visitor=name)

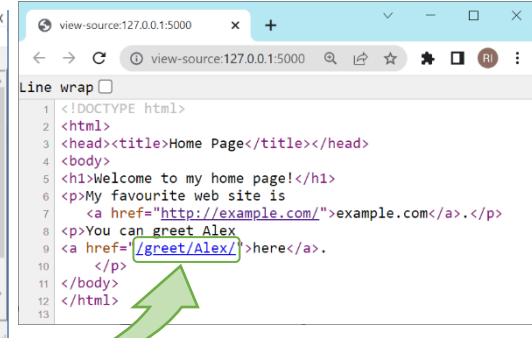
if __name__ == '__main__':
    app.run()
```

- `render_template()` replaces placeholders with the values provided and returns full HTML to the browser.
- Visit `http://127.0.0.1:5000/` and View Source to verify that the template is rendered.
- Visit `http://127.0.0.1:5000/greet/Alex/` and verify that there are no placeholders in its HTML source.

Template home.html



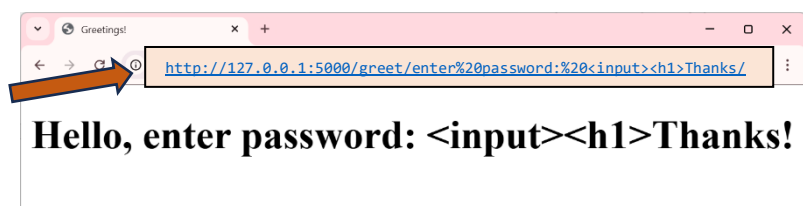
Source of `http://127.0.0.1:5000/`



A green arrow points from the placeholder in the template to the rendered HTML in the browser source.

Rendered HTML is the same as template except placeholders are replaced with the result of Jinja2 expressions

Now when someone **tries to inject HTML**, Jinja2 will automatically escape user input:



Hello, enter password: <input><h1>Thanks!



7.3 Passing Values to Templates

The `render_template()` function takes the template filename in the `templates` subfolder as the first argument, followed by **keyword arguments** to pass values to Jinja2 variables.

Example:

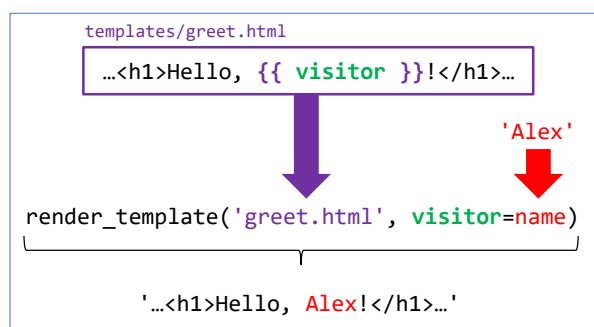
```
return render_template('greet.html', visitor=name)
```

In the template `greet.html`:

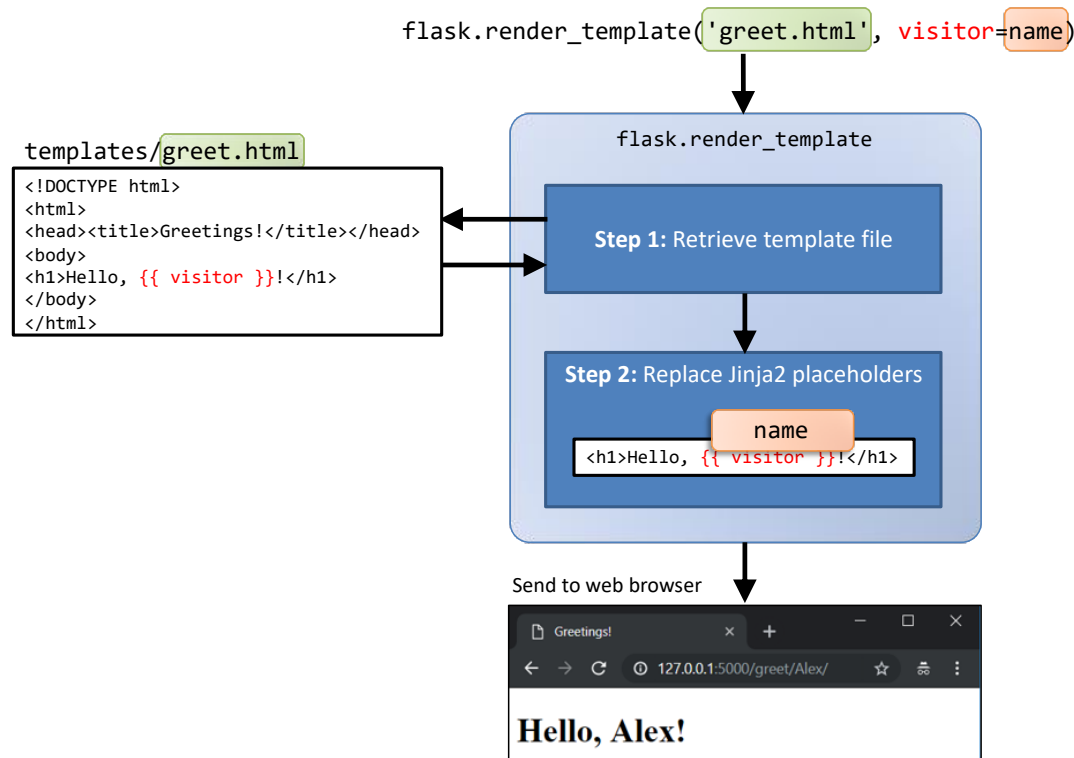
```
<h1>Hello, {{ visitor }}!</h1>
```

`{{ visitor }}` is replaced with the value of `name`.

Do not confuse Jinja2 variables with Python variables. To use Python values in the generated HTML, we must pass them over as keyword arguments when `render_template()` is called. Rendering is the process of **filling placeholders** to produce the final HTML sent to the browser.



The following diagram illustrates how `render_template()` retrieves a template file and replaces its placeholders to produce the final HTML that is sent to the browser:





Exercise 3

Create the following four templates and save them in a templates subfolder:

templates/A.html <pre><!doctype html> <html> <head><title>A</title></head> <body>A</body> </html></pre>	templates/B.html <pre><!doctype html> <html> <head><title>B</title></head> <body>B</body> </html></pre>
templates/C.html <pre><!doctype html> <html> <head><title>C</title></head> <body>C</body> </html></pre>	templates/D.html <pre><!doctype html> <html> <head><title>D</title></head> <body>D</body> </html></pre>

Each template is meant to be shown for a different URL:

URL	Template to use
http://127.0.0.1:5000/A	A.html
http://127.0.0.1:5000/B	B.html
http://127.0.0.1:5000/C	C.html
http://127.0.0.1:5000/D	D.html

Complete the following program accordingly:

app.py

```
from flask import Flask, render_template
app = Flask(__name__)

@
def view_A():
    return render_template('A.html')

@
def view_B():
    return render_template('B.html')

@
def view_C():
    return render_template('C.html')

@
def view_D():
    return render_template('D.html')

if __name__ == '__main__':
    app.run()
```



Generalise the previous program and templates so that the application only needs one template instead of four templates:

templates/template.html

```
<!doctype html>
<html>
<head><title>{{      }}</title></head>
<body>{{      }}</body>
</html>
```

app.py

```
from flask import Flask, render_template
app = Flask(__name__)

def view(      ):
    return render_template('template.html',      )

if __name__ == '__main__':
    app.run()
```

Edit the program above so only the four cases given are allowed and return error code 500 for other cases.

7.4 Avoiding Hardcoded Links

Hardcoding URLs in templates is fragile. Paths must be updated whenever routes change. Instead, links should be generated dynamically using `url_for('function_name', ...)`. This makes templates **more maintainable** and **future-proof**.

```
<p>You can greet Alex
  <a href="{{ url_for('greet', name='Alex') }}">here</a>.
```

Using `url_for()` is more future-proof than hardcoding the path like this:

```
<p>You can greet Alex <a href="/greet/Alex/">here</a>.
```



Exercise 4

Enter the following program:

app.py

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('links.html')

@app.route('/greeting')
def hello():
    return 'Hello, World!'

@app.route('/<int:year>')
def report(year):
    return 'year is ' + str(year)

if __name__ == '__main__':
    app.run()
```

Use the `url_for()` function to complete the following template so that each link will display the respective message when clicked:

templates/links.html

```
<!doctype html>
<html>
<head><title>Links</title></head>
<body>

    <p><a href="{{          }}">Hello, World!</a></p>

    <p><a href="{{          }}">year is 2017</a></p>

</body>
</html>
```



7.5 The safe Filter

By default, Jinja2 **automatically escapes** user input. This means special characters like `<` and `>` are converted to `<` and `>`, which prevents users from injecting HTML or scripts into your page. If you are sure some content is safe and want it rendered as HTML, you can use the **safe filter**:

Template 3: templates/custom.html

```
<!DOCTYPE html>
<html>
<head><title>Custom HTML</title></head>
<body>
<h1>Custom HTML</h1>
{{ my_html|safe }}
</body>
</html>
```

Run the following Python program that makes use of templates/custom.html:

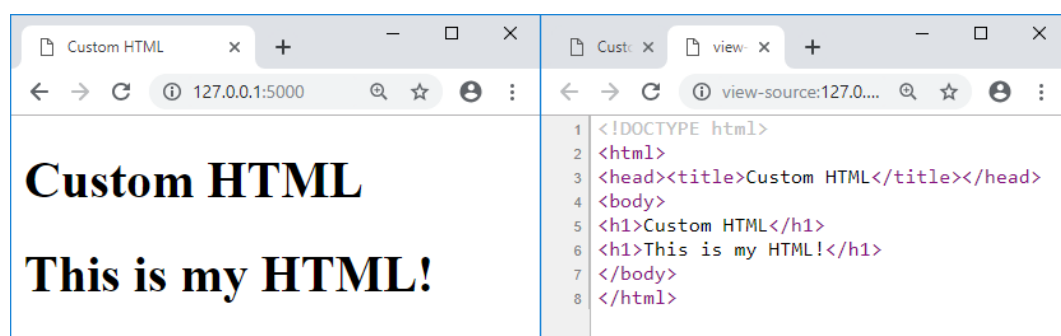
```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('custom.html', my_html='<h1>This is my HTML!</h1>')

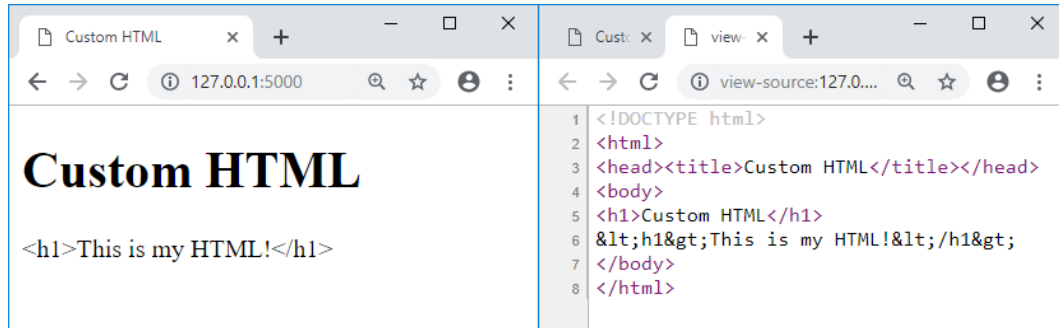
if __name__ == '__main__':
    app.run()
```

Visit <http://127.0.0.1:5000/> and you should see that the string passed to the template as `my_html` is rendered as raw HTML:





However, if we remove the safe filter from custom.html, **my_html** is rendered such that the special characters < and > are escaped, as expected:



Use safe sparingly and only for trusted content.

7.6 The length Filter

Jinja2 does **not support Python functions like len()** directly. Instead, use the length filter to find the length of a string or list.

Template 4: templates/length.html

```
<!DOCTYPE html>
<html>
<head><title>Length of Name</title></head>
<body>
<h1>Length of Name</h1>
Hello {{ name }},
your name is {{ name|length }} characters long!
</body>
</html>
```

Run the following Python program that makes use of templates/length.html:

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/<name>/')
def length_of_name(name):
    return render_template('length.html', name=name)

if __name__ == '__main__':
    app.run()
```

Visit different URLs such as <http://127.0.0.1:5000/vonNeumann/> or <http://127.0.0.1:5000/Ben/> and examine how the length filter is used to determine the number of characters in name.



7.7 Jinja2 Statements

Most logic should be done in Python, not in templates. Jinja2 allows simple control flow using HTML:

```
{% ... %}
```

Unlike placeholders surrounded by double braces `{{` and `}}` that are usually replaced with output, the contents of `{%` and `%}` do not produce any output.

if Statement (Conditional Rendering)

The Jinja2 if statement is used to selectively include or exclude portions of the template.

Template 5: templates/results.html

```
<!DOCTYPE html>
<html>
<head><title>Results</title></head>
<body>
<h1>Results</h1>

{% if greet %}                                <!-- If greet is True, display greeting -->
    <p>Hello, {{ name }}.</p>
{% endif %}

<!-- Conditional display based on score -->
{% if show_score %}                            <!-- If show_score is True, display exact score -->
    <p>Your score is {{ score }}%.</p>
{% elif score >= 50 %}                        <!-- If score is 50 or more -->
    <p>You passed.</p>
{% else %}                                    <!-- If score is below 50 -->
    <p>You failed.</p>
{% endif %}

</body>
</html>
```

Run the following program and visit <http://127.0.0.1:5000/> to see the template rendered for a score of 72:

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('results.html', greet=True, name='Alex',
                           show_score=False, score=72)

if __name__ == '__main__':
    app.run()
```



for-in Statement (Looping)

The Jinja2 **for-in** statement is used to repeat the rendering of a portion of the template for every item in a list, tuple, string or dictionary.

A typical use of **for-in** is to output the contents of a collection in a table:

Template 6: templates/table.html

```
<!DOCTYPE html>
<html>
<head><title>Table of Results</title></head>
<body>
<h1>Table of Results</h1>

<table>
<tr><th>Subject Name</th><th>Score</th></tr>
{% for subject in results %}  <!-- iterates through each subject in results -->
    <tr>
        <td>{{ subject }}</td>          <!-- Display subject name -->
        <td>{{ results[subject] }}</td> <!-- Display score of subject -->
    </tr>
{% endfor %}  <!--End of loop -->
</table>

</body>
</html>
```

For this example, table.html expects a Jinja2 variable named results containing a dictionary mapping subject names to scores. The following program creates a hardcoded dictionary matching this format and passes it to the template for rendering:

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    results = {
        'English': 75,
        'Mother Tongue': 73,
        'Maths': 76,
        'Computing': 78
    }
    return render_template('table.html', results=results)

if __name__ == '__main__':
    app.run()
```

Run the following program and visit <http://127.0.0.1:5000/> to see how the hardcoded data in the dictionary is rendered as a HTML table. In a real web application, we would retrieve the subject names and scores from an SQLite database instead of hardcoding them.



8. Static Files

Static files are resources that **do not change**, e.g. CSS (stylesheets), images.

Flask stores these in a **folder named static/**.

Files in this folder are accessed using **paths starting with /static/**.

Create the following css file and save as `static/styles.css`:

Style Sheet 1: `static/styles.css`

```
body {  
    background: yellow;  
}  
  
h1 {  
    border-bottom: 1px solid red;  
    color: red;  
}
```

Now create a HTML file that links to this style sheet. The URL of the style sheet can be hardcoded `/static/styles.css`. However, it is preferred to use `url_for()` to link static files.

Template 7: `templates/stylish.html`

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Stylish Page</title>  
    <link  
        rel="stylesheet"  
        href="{{ url_for('static', filename='styles.css') }}">  
    <!--  
        The above can also be written as:  
    <link rel="stylesheet" href="/static/styles.css">  
    -->  
</head>  
<body>  
    <h1>Stylish Page</h1>  
    <p>Look at how stylish this page is!</p>  
</body>  
</html>
```

Note how the path of the static file within the static subfolder is passed to `url_for()` as a keyword argument named `filename`.



Finally, write a Python program to render this template:

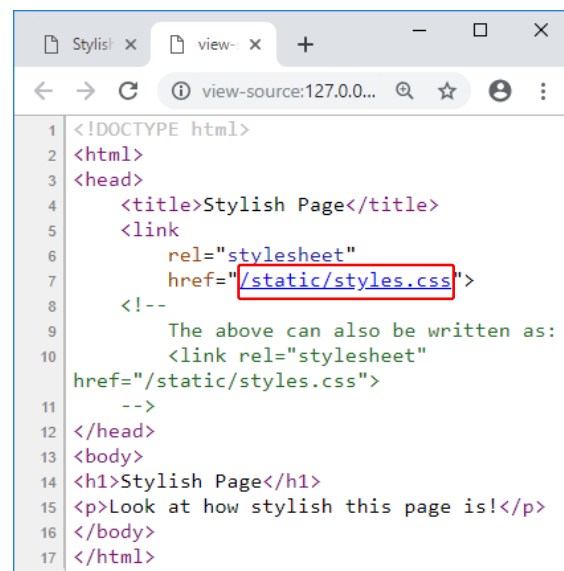
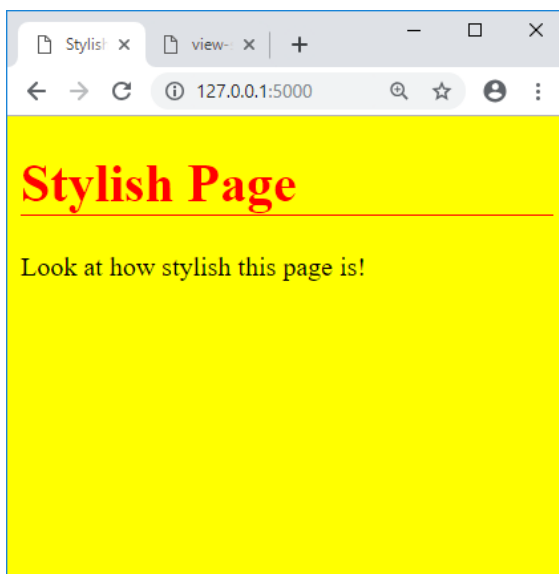
```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('stylish.html')

if __name__ == '__main__':
    app.run()
```

Visit `http://127.0.0.1:5000/` with this program running and you should see the styled page. Use the View Source feature by pressing Ctrl-U to verify that the style sheet URL placeholder is replaced with a path starting with `/static/`:





9. Processing Form Data

Users usually send input through **HTML forms**. Form data is sent as **key-value pairs** to the server. Flask can access this data using the request object.

9.1 GET Requests

The default method for HTML forms is to submit a GET request. Data is sent in the URL (query string).

For instance, the URL `http://example.com/hello?name=bala&age=18` contains a query with two key-value pairs: one for the key 'name' and another for 'age'.

Flask stores this data in `request.args`.

Create these two HTML files – one to capture input, another to display results.

Template 8: templates/form_with_get.html

```
<!DOCTYPE html>
<html>
<head><title>GET Form</title></head>
<body>
<form action="{{ url_for('process_with_get') }}" method="get">
  <p>Input s: <input name="s"></p>
  <p><input type="submit"></p>
</form>
</body>
</html>
```

Template 9: templates/analysis_results.html

```
<!DOCTYPE html>
<html>
<head><title>String Analysis</title></head>
<body>
<p>You entered s: {{ s }}</p>
<p>This string has
  {{ num_vowels }} vowels(s) and
  {{ num_words }} word(s).
</p>
</body>
</html>
```



Next, write a Flask application with two routes: the root `/` route simply renders the form while the `/process/` route is associated with a Python function named `process_with_get()` that analyses `s` and renders the result:

```
from flask import Flask, render_template, request

VOWELS = ['a', 'e', 'i', 'o', 'u']

app = Flask(__name__)

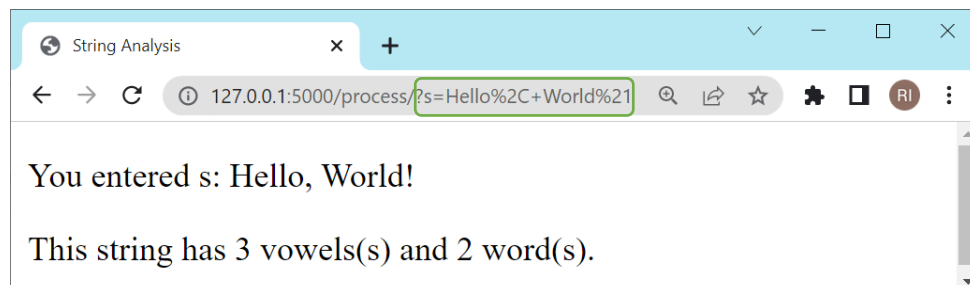
@app.route('/')
def form():
    return render_template('form_with_get.html')

@app.route('/process/')
def process_with_get():
    if 's' in request.args:
        s = request.args['s']
        lower_s = s.lower()
        num_vowels = 0
        for vowel in VOWELS:
            num_vowels += lower_s.count(vowel)
        num_words = len(s.split())
        return render_template('analysis_results.html', s=s,
                               num_vowels=num_vowels, num_words=num_words)
    return 'No form data found!'

if __name__ == '__main__':
    app.run()
```

- GET sends data in the **URL**
- Access data using **request.args**
- Data is stored as **key-value pairs**
- Always check if key exists (`'s' in request.args`)

Visit `http://127.0.0.1:5000/` and fill in the form. When you submit the form, notice that the value of `s` you entered is encoded in the resulting URL.



It is possible to visit `http://127.0.0.1:5000/process/` without filling in the form. If you do so, the `request.args` dictionary will not include a value for `'s'` and the error message (`'No form data found!'`) will be returned instead.



9.2 POST Requests

Submitting form data using a GET request has several disadvantages:

1. Submitted form data is recorded in the resulting URL, which is viewable in the browser history.
2. GET requests are not supposed to make changes to the server's data. By using a GET request to add, delete or update data from a database, we are not following the HTTP standard.
3. Some browsers and server software limit the length of URLs, so there is a risk that overly long form data submitted using GET may get truncated.

The POST request is used to overcome these disadvantages. POST is used to send form data **securely**. Data is **not visible in the URL**. It can be used to modify server data. There is no URL length limitations.

Create the following HTML file:

Template 10: templates/form_with_post.html

```
<!DOCTYPE html>
<html>
<head><title>POST Form</title></head>
<body>
<form method="post" action="/">                                <!-- Sends data using POST method -->
    <p>Input s: <input name="s"></p>
    <p><input type="submit"></p>
</form>
</body>
</html>
```

The **action** attribute is removed so the form is submitted to the **same URL** used to generate the form.

Now, write a Flask application with both GET and POST methods in the root / route.

```
from flask import Flask, render_template, request

VOWELS = ['a', 'e', 'i', 'o', 'u']

app = Flask(__name__)

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'GET':
        return render_template('form_with_post.html')
    #If the form contains a field called s AND it is not empty.
    if 's' in request.form and request.form['s']:
        s = request.form['s']
        lower_s = s.lower()
        num_vowels = 0
        for vowel in VOWELS:
```



```
        num_vowels += lower_s.count(vowel)
    num_words = len(s.split())
    return render_template('analysis_results.html', s=s,
                           num_vowels=num_vowels, num_words=num_words)

    return 'No form data found!'

if __name__ == '__main__':
    app.run()
```

Here, `index()` function **checks if the request method** is 'GET', and if so, simply renders the form and exits. Otherwise, the request method is assumed to be 'POST' and the submitted form data is processed instead.

If there is no value for 's' in `request.form`, an error message ('No form data found!') is returned.

Differences between GET and POST methods:

Feature	GET	POST
Data location	URL	Request body
Visibility	Visible	Not visible
Use	Retrieve data	Submit/modify data
Flask access	<code>request.args</code>	<code>request.form</code>



Exercise 5

Enter the following and save it as a template named `form.html`:

`templates/form.html`

```
<!doctype html>
<html>
<head><title>Greeting Form</title></head>
<body>
  <form method="POST" action="/">
    <input type="text" name="name">
    <input type="text" name="address">
    <input type="submit">
  </form>
  
  form.html
</body>
</html>
```

Complete the following program and template so visiting `http://127.0.0.1:5000/` will render **form.html** and submitting the form will display “Hello *name*, your address is: *address*”, where *name* and *address* are the content of the first and second text field respectively:

`templates/results.html`

```
<!doctype html>
<html>
<head><title>Greeting</title></head>
<body>
  Hello {{      }}, your address is: {{      }}
</body>
</html>
```

`app.py`

```
from flask import
app = Flask(__name__)

# Use the space below


if __name__ == '__main__':
    app.run()
```



10. Handling File Uploads

Web applications often allow users to upload files. HTML forms can support file uploads using:

```
<input type="file" name="file">
```

Requirements for file upload forms:

- The form must use the **POST** method:
`<form method="post">`
- The form must include the **enctype** attribute set to **"multipart/form-data"**:
`<form method="post" enctype="multipart/form-data">`

This allows the browser to send the file data to the server.

Create a HTML template for uploading files:

Template 11: templates/upload.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Upload File</title>
</head>
<body>
    <h1>Upload a File</h1>
    <!-- Form to upload a file -->
    <form method="post" action="/" enctype="multipart/form-data">
        <input type="file" name="file">
        <input type="submit" value="Upload">
    </form>
    <!-- List uploaded files -->
    {% if files %}
        <h2>Uploaded Files</h2>
        <ul>
            {% for f in files %}
                <!-- Link to view uploaded files -->
                <li> <a href="{{ url_for('uploads', filename=f) }}">{{ f }}</a> </li>
            {% endfor %}
        </ul>
    {% endif %}
</body>
</html>
```

- The `files` variable will contain the list of uploaded filenames.
- Each file is displayed as a link to view/download it.



Write a Flask application for file uploads:

```
import os
from flask import Flask, request, render_template, send_from_directory

app = Flask(__name__)

UPLOAD_FOLDER = 'uploads' # Folder to store uploaded files
os.makedirs(UPLOAD_FOLDER, exist_ok=True) # Create folder if it doesn't exist

@app.route('/', methods=['GET', 'POST'])
def upload_file():
    """
    Handles file uploads and display the gallery:
    - GET: Retrieves list of existing files and renders the upload page
    - POST: Processes uploaded file, saves it to server, and refreshes the list
    """
    if request.method == 'POST' and 'file' in request.files:
        f = request.files['file'] # Extract the file object from the request
        if f.filename != '': # Ensure user actually selected a file
            # Join the folder path and filename to save the file
            f.save(os.path.join(UPLOAD_FOLDER, f.filename))

        files = os.listdir(UPLOAD_FOLDER) # Retrieve list of all uploaded files
        return render_template('upload.html', files=files)

@app.route('/uploads/<filename>')
def uploads(filename):
    """
    Serves a specific file from the upload directory to the browser.
    Used for viewing or downloading previously uploaded content.
    """
    return send_from_directory(UPLOAD_FOLDER, filename)

if __name__ == '__main__':
    app.run()
```

- **request.files**
 - Contains all files submitted via a form with enctype="multipart/form-data".
 - Access a file using: `f = request.files['file']`.
 - Save the file to the server with `f.save(path)`.
- Always check if `f.filename != ''` to ensure a file was selected.
- Uploaded files can be served using `send_from_directory(folder, filename)`.
- The uploaded files are stored in the 'uploads' folder (create it before running the app).
- Visit `http://127.0.0.1:5000/` to view the upload form.
- Upload a file.
- The uploaded file appears in a list below the form. Click the link to view it.



11. Summary of Request Data in Flask

Attribute	Purpose	HTTP Method	Typical Use	Example
<code>request.args</code>	Access data submitted in the URL query	Usually GET, but also works with POST if URL has query portion	Reading form/query parameters	If you have a URL like <code>http://abc.com/search?query=Flask</code> , you can use <code>request.args['query']</code> to access the value 'Flask' submitted with the query parameter.
<code>request.form</code>	Access form data submitted from a web page	POST only	Reading submitted form fields	If you have a form with a field named 'username', you can access the value submitted with that field using <code>request.form['username']</code> .
<code>request.files</code>	Access uploaded files from a form	POST only	Saving or processing uploaded files	If you have a form with a file input field named 'file', you can access the uploaded file using <code>request.files['file']</code> .



12. Programming Exercises

TASK 1: Hospital

Task 1.1

Design a web page for managing patient information with the following requirements:

- Choose a background colour that is not white and text colour that is not black.
- Include a main heading “Hospital Appointment System” and a subheading “Search Patient Record”.
- Place all elements inside a centred container (*div*) with padding, a visible border, and a background colour different from the page.
- Use a `<form>` with the POST method containing a labelled input for `PatientID` and a Submit button.
- Style the button to change colour on hover (*button:hover { }*)
- Save this web page as: `hospital_form.html`
- Sample output:

Hospital Appointment System

Search Patient Record

Patient ID:

[Submit](#)

Task 1.2

Design a web page to display the patient’s information retrieved from the database.

- The `PatientID` must be reflected in the heading: “Patient Record: `<PatientID>`”.
- Display information in a **bordered table** with the following fields retrieved from the database: *PatientID, Name, Email, Gender, Date of Birth, Last Visit Date, Number of Appointments, Has Insurance (Yes/No), and Outstanding Bills (\$)*.
- The table must include visible borders, cell padding, and a distinct background colour for the header row.
- Include a “Go Back” button at the bottom that returns the user to the root route (/).
- Use the same colour theme as Task 1.1.
- Save this web page as: `hospital_display.html`



Sample output:

Patient Record: P001

Field	Details
PatientID	P001
Name	Billy Tan
Email	billytan@email.com
Gender	Male
Date of Birth	01-10-1999
Last Visit Date	15-03-2026
Number of Appointments	5
Has Insurance	No
Outstanding Bills (\$)	0

[Go Back](#)

Task 1.3

Using the **Jupyter Notebook** platform, write a Python program that makes use of the `hospital_form.html` and `hospital_display.html` to create a web application that:

- receives a `PatientID` string from the `hospital_form.html` form, then
- returns the `hospital_display.html` web page to display a table showing all the information associated with the `PatientID` input.

Your program should make use of the SQL database `hospital.db`.

Save the program code as `TASK1_3_<your_name>_<class>.ipynb`.

Remember to place the two HTML files inside a folder named **templates** in the same directory as your Jupyter Notebook, otherwise, Flask will not be able to find them.

Run the web application and input the following `PatientID` and save the output as a html file:

Input PatientID	Filename of web page output
P013	<code>TASK1_3A_<your_name>_<class>.html</code>
P017	<code>TASK1_3B_<your_name>_<class>.html</code>



TASK 2: Bakery (adapted from 2020 HCI P2Q3)

A bakery currently keeps records on paper of all its products it sells in the shop. It decided to trial a database to manage its products. It is expected that the database should be normalized.

The following information of the bakery is recorded:

- `ProductCode` – Unique code of the item
- `Name` – Name of the item
- `Type` – The type of product – 'Cake', 'Loaf', 'Bun'
- `Location` – The location at which the product is made – 'North', 'South', 'East', 'West' kitchen
- `Price` – The selling price of the product

For cakes, the following additional information is recorded:

- `ServingSize` – the estimated number of servings per cake
- `Shape` – Shape of the cake – 'Square', 'Circle', 'Roll'

For loaves, the following additional information is recorded:

- `Weight` – weight of loaf in gram

For buns, the following additional information is recorded:

- `PiecesPerPackage` – number of pieces per package

The information is to be stored in four different tables:

- `Product`
- `Cake`
- `Loaf`
- `Bun`

Task 2.1: Create database and 4 tables

Create a SQL file named `Task2_1_<your_name>_<class>.sql` to show the SQL code to create the database **bakery.db** with the four tables.

The table, `Product`, must use the `ProductCode` as its primary key. The other tables must refer to the `ProductCode` as a foreign key.

Save your SQL code as `Task2_1_<your_name>_<class>.sql`

Task 2.2: Insert information from 3 text files into SQL database

The files `CAKES.TXT`, `LOAVES.TXT` and `BUNS.TXT` contain information about the bakery's cakes, loaves and buns respectively for insertion into the bakery database.



RAFFLES INSTITUTION H2 Computing (9569)

2026 Year 6

Each row in the three files is a comma-separated list of information about a single product.

For `CAKES.TXT`, the information about each cake is given in the following order:

- `ProductCode`, `Name`, `Location`, `Price`, `ServingSize`, `Shape`

For `LOAVES.TXT`, the information about each loaf is given in the following order:

- `ProductCode`, `Name`, `Location`, `Price`, `Weight`

For `BUNS.TXT`, the information about each bun is given in the following order:

- `ProductCode`, `Name`, `Location`, `Price`, `PiecesPerPackage`

Write a Python program to insert all information from the three files into the bakery database, `bakery.db`. Run the program.

Save your program code as `Task2_2_<your_name>_<class>.py`

Task 2.3: SQL for Circle cake

Write SQL code to show the `ProductCode`, `Name`, `Location`, `Price` and the `ServingSize` of each cake with 'Circle' shape.

Run the query. Save this code as `Task2_3_<your_name>_<class>.sql`

Task 2.4: Filter by Location and display in web browser

The bakery wants to filter the products by `Location` and display the results in a web browser.

Write a Python program and the necessary files to create a web application that:

- receives a `Location` string from a HTML form, then
- creates and returns a HTML document that enables the web browser to display a table tabulating the details of the product based on the `Location` in ascending order of `Price`. The table will display the following columns: `Name`, `Type`, `Price`.

Save your Python program as `Task2_4_<your_name>_<class>.py` with any additional files/sub-folders as needed in a folder named `Task2_4_<your_name>_<class>.`

Run the web application.

Save the output of the program when 'North' is entered as the `Location` as `Task2_4_<your_name>_<class>.html`



TASK 3: Enrichment Day Activities (adapted from 2020 NYJC P2Q4)

A school wants to create a database to allow students to register for different enrichment activities that will be held on the school's Enrichment Day. An enrichment activity falls under one of three categories – Arts, Cultural, and Sports.

It is expected that the database should be normalised.

When a student registers for an activity, the following information is recorded:

- `StudentID` - unique 6-digit register number of the student.
- `Type` - type of activity ('A', 'C' or 'S').
- `Venue` - where the activity will be held.
- `Session` - whether the activity is conducted in the morning or afternoon ('AM' means the morning session, and 'PM' means the afternoon session).

For the Arts category, the following extra information is recorded:

- `Performance` – “True” for performance arts, “False” for visual arts.

For Cultural, the following extra information is recorded:

- `Race` – which race the culture belongs to.

For Sports, the following extra information is recorded:

- `Contact` – “C” to denote contact sports, “NC” to denote non-contact sports.
- `Cost` - the amount of money in dollars (not more than \$50) the student must pay the instructor.

The information is to be stored in four different tables:

- `Registration`
- `Arts`
- `Cultural`
- `Sports`

Task 3.1: Create database and 4 tables

Create an SQL file called `TASK3_1_<your name>.sql` to show the SQL code to create the database `register.db` with the four tables. The table, `Registration`, must use `StudentID` as its primary key. The other tables must refer to the `StudentID` as a foreign key.

Save your SQL code as `TASK3_1_<your name>.sql`.



Task 3.2: Create HTML file

The school wants to allow students to register over the internet. The form design for the web page is shown below:

StudentID:	
Type ('A', 'C' or 'S'):	
Venue:	
Session:	
Fill in the extra detail(s) for the chosen type of activity:	
Arts	
Performance (True) or Visual Arts (False):	
Cultural	
Race:	
Sports	
Contact (C or NC):	
Cost:	
Register	

Students can enter their details and click "Register" button to insert their data into the database.

Design the web page as shown above, using suitable colours for background and text. Name this HTML file `start.html`.

Task 3.3: Create Python program to register details

Write a Python program to create a web application that:

- accepts input from the web page `start.html` (assume input is keyed in correctly, no validation needed), and
- inserts the registration details into `register.db` when "Register" button is clicked.

Save your Python program as `TASK3.3_<your name>_<class>.py`

Test your web application by entering the following records via the form's submit button:

```
192701, A, MPH, AM, True
192703, A, ISH, PM, False
192723, S, Field, AM, C, 20
192803, C, LT3, AM, Malay
192820, S, CourtA, PM, NC, 15
193005, C, LT4, PM, Chinese
193006, C, LT4, PM, Chinese
```



Task 3.4: Generate AM/PM Session

Add two buttons to your `start.html` web page that will generate the AM and PM session activities respectively upon clicking.



Upon clicking either the generate AM or PM session buttons, another web page `result.html` will be loaded that will display all the details of either the AM or PM session activities.

Design this web page `result.html` and add to your Python program code for the web application that will generate the web page `result.html`.

Name the updated Python program as `TASK3_4_<your name>_<class>.py`

You should submit 3 files:

- `start.html`,
- `result.html` and
- `TASK3_4_<your name>_<class>.py`